Software Atelier 1 — Group Project

How to contribute to the GitHub repository

Edoardo Beretta

Latest update: November 3, 2026

We are about 20 people working on the same repo, so we can't all push directly to `master` like in the hands-on. Hence, the branch `master` is protected: nobody can push to it directly. Every change goes through a *branch* and a *pull request*.

The basics (`clone`, `add`, `commit`, `push`, `pull`, conflicts) work exactly like we saw in class. This document only explains branches and pull requests, and what you have to do step by step.

1 Branches and pull requests

A **branch** is a separate line of commits. `master` is the main branch: it's the shared version of the site, the one that is online, and it must always work.

When you create a branch, it starts as a copy of `master` at that moment. All the commits you make go on your branch only, so you can work without touching `master` and without getting in the way of the others. Nobody sees your work until you push the branch.

When your work is done, you open a **pull request** (PR) on GitHub: it's a request to merge your branch into `master`. GitHub shows exactly which lines you changed, someone else reviews them and approves, and then the branch is **merged** into `master`. At that point your changes are part of the shared version.

2 Setup (only to do once)

1. Accept the invitation to the repository (you get it by email or in your GitHub notifications). Without it you can't push.

2. Clone the repo, like in the hands-on:

   ```
   git clone https://github.com/imdaliia/swat-g1.git
   ```

3. Configure git, like in the hands-on (if you already did it, nothing changes):

   ```
   git config --global user.name "Name Surname"
   git config --global user.email "you@example.com"
   git config --global pull.rebase false
   ```

   Use the same email as your GitHub account. The last line makes `git pull` merge instead of rebase, like we saw in class.

Clone only **once**. For each new task you create a new branch, not a new clone.

3 What to do for every task

1. Go on `master` and get the latest version:

   ```
   git checkout master
   git pull
   ```

2. Create your branch and move onto it (`-b` creates it):

   ```
   git checkout -b yoursurname/what-you-do
   ```

   For example `beretta/contrib-docs` or `berti/global-css`: lowercase, no spaces, words separated by `-`.

3. Work on the files, then add and commit as usual:

   ```
   git add style.css
   git commit -m "edit: menu colors"
   ```

   You can make as many commits as you want. Before committing, check with `git status` that you are on your branch and not on `master`.

4. Push your branch. The first time you need `-u origin` and the branch name, after that just `git push`:

   ```
   git push -u origin yoursurname/what-you-do
   ```

5. Open the pull request: on the repo page GitHub shows a *Compare & pull request* button right after the push. Write a short title and what you changed as description, then create it and send the link in the group chat.

6. Wait for someone to review it. If they ask for changes, fix them *on the same branch*, commit and `git push` again: the PR updates by itself, don't open a new one.

7. Once it's approved, it gets merged into `master` (don't force merge your own PR). The demo website updates by itself within a couple of minutes.

8. For the next task, start again from step 1 with a new branch.

4 When to create a branch

- One branch for each task, **not one branch for each person**. A task is something you can describe in one short sentence, like "add the contribution docs" or "modify the global css".
- Keep them small. A PR with a few files is reviewed in five minutes, one with thirty files sits there for days.
- Don't mix different things in the same branch. If the menu fix is fine but the new colors aren't, the whole PR is blocked.
- Never reuse a branch after it's been merged. Next task, new branch from an updated `master`.
- Always create the branch from an updated `master` (step 1), otherwise you start from an old version of the site.

If you know someone else is working on the same part of a file, talk to them first and decide who goes first. It saves you a conflict.

5 If master changed in the meantime

While you work, other PRs get merged into `master`. Usually you don't have to do anything. You need to bring `master` into your branch only if GitHub says that your PR has conflicts:

```
git checkout master
git pull
git checkout yoursurname/what-you-do
git merge master
```

If there are conflicts, you solve them like in class: open the file, keep the right version between the `<<<<<<<`, `=======` and `>>>>>>>` markers, remove the markers, then `git add` and `git commit`. Then `git push`.

6 If something goes wrong

You committed on master by mistake. Your push will be rejected (`GH013: Repository rule violations`). Nothing is lost: move your commits to a new branch and put `master` back as it is on GitHub:

```
git checkout -b yoursurname/what-you-do
git push -u origin yoursurname/what-you-do
git checkout master
git reset --hard origin/master
```

Then open the PR as usual. Use `reset --hard` only in this case.

Anything else. Don't try random commands from the internet and never use `--force`. Run `git status`, take a screenshot and ask in the group chat.